

```

// A manifest that declares a schema gets a working module for free: create,
// list, update, remove, each with a verifier, plus a list renderer.
// This is what makes "describe it and it exists" real without writing code.
export function moduleFor(manifest) {
  const key = manifest.key;
  const entity = Object.keys(manifest.schema ?? {})[0];
  if (!entity) return { capabilities: [], renderers: {} };

  const fields = Object.keys(manifest.schema[entity].fields);

  const pick = (input) => {
    const row = {};
    for (const f of fields) if (input[f] !== undefined) row[f] = input[f];
    return row;
  };

  const capabilities = [
    {
      key: `${key}.create`,
      summary: `Add a ${entity}.`,
      route: 'internal',
      handler: async ({ input, gateway }) => gateway.insert(entity, pick(input)),
      verify: async ({ result, gateway }) => {
        const [row] = await gateway.select(entity, { where: { id: result.id } });
        return { state: row ? 'verified' : 'failed', evidence: [{ kind: 'row', id
      },
    },
    {
      key: `${key}.list`,
      summary: `List every ${entity}.`,
      route: 'internal',
      handler: async ({ gateway }) => gateway.select(entity, {}),
      verify: async ({ result }) => ({
        state: 'verified', evidence: [{ kind: 'row_count', actual: result.length
      },
    },
    {
      key: `${key}.update`,
      summary: `Change a ${entity}.`,
      route: 'internal',
      handler: async ({ input, gateway }) => gateway.update(entity, input.id, pic
      verify: async ({ input, gateway }) => {
        const [row] = await gateway.select(entity, { where: { id: input.id } });
        return { state: row ? 'verified' : 'failed', evidence: [{ kind: 'read_bac

```

```

    },
  },
  {
    key: `${key}.remove`,
    summary: `Remove a ${entity}.`,
    route: 'internal',
    handler: async ({ input, gateway }) => gateway.remove(entity, input.id),
    verify: async ({ input, gateway }) => {
      const [row] = await gateway.select(entity, { where: { id: input.id } });
      return { state: row ? 'failed' : 'verified', evidence: [{ kind: 'absent',
    },
  },
];

return {
  capabilities,
  renderers: {
    list: async ({ gateway }) => ({ items: await gateway.select(entity, {}) }),
  },
};
}

```